

Especificação Técnica - App de Análise e Preparação de Ordens IBKR

Documento técnico de arquitetura, dados, integrações, segurança e fases de implementação

Nota: este documento é uma especificação de trabalho. Deve ser atualizado à medida que as decisões técnicas forem fechadas e que as fases forem implementadas.

Versão: 0.1 | Estado: proposta técnica inicial | Público-alvo: equipa de desenvolvimento e decisores do projeto

1. Objetivo do sistema

Construir uma aplicação local, de utilização controlada, para analisar mercados, testar estratégias em dados históricos, gerar sinais explicados e preparar ordens para a Interactive Brokers. A aplicação começa em modo simulação/paper e não executa ordens sem confirmação humana explícita por ordem.

- O sistema informa, simula, explica e prepara.
- O utilizador confirma manualmente cada ordem antes de transmissão.
- A primeira validação de qualquer estratégia acontece em backtesting e paper trading.
- A app não promete rentabilidade e não deve ser tratada como robô autónomo de trading.

2. Princípios não negociáveis

- Paper trading é o modo inicial e o modo por defeito.
- Não existe auto-execução de ordens, confirmação em lote ou loop autónomo que coloque ordens sem intervenção humana.
- Um sinal nunca é uma ordem. Um sinal apenas sugere uma possível ação.
- Uma ordem só pode ser transmitida depois de passar pelo RiskEngine e depois de confirmação humana específica.
- Credenciais da IBKR nunca ficam guardadas no código, na base de dados ou no frontend.
- Todo o evento relevante fica registado em audit trail: sinais, ordens preparadas, confirmações, rejeições, transmissões, fills, erros e alterações de modo.
- O kill switch deve estar sempre acessível e bloquear imediatamente novas preparações de ordem.
- Qualquer passagem para live deve exigir fricção deliberada: alteração de configuração, banner persistente e ecrã de reconhecimento de risco.

3. Arquitetura geral

A arquitetura deve separar claramente análise, simulação, risco, ordens e integração com broker. A estratégia não deve saber se está a correr em backtest, paper ou live; deve apenas receber dados e devolver sinais.

Frontend React

- > FastAPI Backend
 - > MarketDataService
 - > IndicatorEngine
 - > StrategyEngine
 - > BacktestingEngine
 - > RiskEngine
 - > OrderPreparationService

- > AuditService
- > PostgreSQL
- > IBKRAdapter -> IB Gateway/TWS -> Interactive Brokers

4. Stack tecnológica

Área	Tecnologia	Função
Backend	Python 3.11+	Lógica de mercado, APIs locais, backtesting, risco, integrações e persistência.
API local	FastAPI + Uvicorn	Servidor local que expõe endpoints para o frontend e coordena os serviços internos.
Dados/análise	pandas	Tratamento de dados OHLCV, cálculo de indicadores e preparação de datasets.
Validação	Pydantic	Modelos de dados seguros para sinais, ordens, configurações e respostas da API.
Base de dados	PostgreSQL	Persistência robusta de candles, sinais, backtests, ordens, fills, auditoria e exportações.
Migrações	Alembic	Gestão versionada da estrutura da base de dados.
ORM	SQLAlchemy	Camada Python para acesso organizado à base de dados.
Logs	structlog	Logs estruturados e legíveis, separados do audit trail transacional.
IBKR	ib_async	Biblioteca Python para comunicar com IB Gateway/TWS via API oficial.
Frontend	React + TypeScript + Vite	Dashboard visual, gráficos, painéis de sinais e fluxos de confirmação.
Gráficos	TradingView lightweight-charts	Renderização de candles, indicadores, marcas de compra/venda e equity curves.
Ambiente local	Docker Compose	Arranque consistente de PostgreSQL e serviços necessários em desenvolvimento.
Testes	pytest + testes frontend	Garantia de comportamento em indicadores, estratégias, risco, backtesting e endpoints.

5. Modelo de dados PostgreSQL

A base de dados deve ser desenhada para auditoria, análise histórica, comparação de backtests e rastreabilidade. O frontend não deve escrever diretamente em tabelas críticas; todas as escritas passam pelo backend.

Tabela	Responsabilidade	Campos principais
instruments	Ativos negociáveis/analísáveis	symbol, exchange, currency, asset_type, ibkr_con_id
market_bars	Velas históricas e intraday	instrument_id, timeframe, ts, open, high, low, close, volume, source, data_type
strategies	Estratégias disponíveis	name, description, enabled
strategy_versions	Versões e parâmetros base das estratégias	strategy_id, version, code_hash, default_params_json
signals	Sinais gerados pelas estratégias	strategy_version_id, instrument_id, ts, direction, strength, rationale_json, indicator_snapshot_json
backtest_runs	Execuções de simulação histórica	strategy_version_id, params_json, start_ts, end_ts, starting_cash, commission_model, slippage_model
backtest_trades	Trades simulados dentro de um backtest	backtest_run_id, entry_ts, exit_ts, side, qty, entry_price, exit_price, pnl

backtest_metrics	Métricas agregadas de backtest	total_return, max_drawdown, win_rate, profit_factor, sharpe, exposure
risk_limits	Limites configuráveis de risco	max_order_notional, max_position, max_open_positions, max_daily_loss, require_stop
prepared_orders	Ordens preparadas mas ainda não transmitidas	source_signal_id, side, qty, order_type, limit_price, stop_price, status
order_confirmations	Confirmações/rejeições humanas	prepared_order_id, user_action, confirmed_at, confirmation_text
broker_orders	Ordens enviadas para IBKR/paper	prepared_order_id, broker_order_id, mode, status
fills	Execuções/fills devolvidos pela corretora	broker_order_id, fill_ts, qty, price, commission, currency
positions_snapshots	Fotografias periódicas da carteira	instrument_id, ts, qty, avg_cost, market_value, unrealized_pnl
audit_log	Registo append-only de eventos críticos	event_ts, actor, event_type, entity_type, entity_id, payload_json
fx_rates	Taxas de câmbio usadas em exportações	date, from_ccy, to_ccy, rate, source
tax_exports	Histórico de exportações fiscais	year, generated_at, format, file_path, fx_source

6. Backend - módulos principais

ConfigService

Carrega .env/config.toml, portas IBKR, modo paper/live, limites de risco e símbolos vigiados.

MarketDataService

Obtém e normaliza dados históricos/live. Mantém rolling OHLCV por símbolo e timeframe.

IndicatorEngine

Calcula indicadores de forma pura e testável: SMA, EMA, RSI, MACD, Bollinger Bands, ATR, VWAP e volume.

StrategyEngine

Executa estratégias configuráveis sobre dados enriquecidos por indicadores e gera Signals.

BacktestingEngine

Simula estratégias em períodos passados, usando os mesmos indicadores e estratégia do modo live.

RiskEngine

Valida qualquer ordem antes de ser mostrada como confirmável.

OrderPreparationService

Transforma uma intenção ou sinal escolhido numa ordem preparada, com payload de confirmação.

IBKRAdapter

Único módulo que importa ib_async e fala com IB Gateway/TWS.

AuditService

Escreve eventos críticos em audit_log de forma append-only.

TaxExportService

Gera CSV/XLSX anual em EUR a partir de trades realizados e taxas FX guardadas.

7. Frontend - ecrãs e componentes

- Dashboard principal: modo atual, estado da ligação, watchlist, posições, sinais recentes e botão de kill switch.
- Gráfico de ativo: candles, indicadores, volume, sinais de entrada/saída e tipo de dados recebido.
- Painel de backtesting: seleção de símbolo, estratégia, parâmetros, período, capital inicial, comissões e slippage.
- Resultados de backtest: equity curve, drawdown, lista de trades, métricas e avisos sobre overfitting.
- Painel de sinais: direção, força, racional, snapshot de indicadores e opção de preparar ordem.
- Modal de confirmação: mostra todos os detalhes da ordem, risco, custos estimados, posição resultante e checks passados.
- Painel de risco: limites ativos, perdas do dia, número de posições e estado do kill switch.
- Exportação fiscal: ano, moeda, ficheiros gerados e aviso de que o relatório oficial IBKR continua a fonte de verdade.

8. Integração IBKR

A ligação à Interactive Brokers deve ser feita pela API oficial através de IB Gateway ou TWS. A preferência operacional é IB Gateway por ser mais leve. O login é feito fora da app, no programa da IBKR; a app abre apenas uma ligação socket para uma sessão já autenticada.

App local -> IBKRAdapter -> ib_async -> IB Gateway/TWS -> Interactive Brokers

- Portas devem vir de configuração. Valores habituais: Gateway paper 4002, Gateway live 4001, TWS paper 7497, TWS live 7496.
- A app deve detetar quedas de ligação e tentar reconexão com backoff.
- A reautenticação semanal/2FA deve gerar banner claro no frontend.
- Os dados de mercado da API podem estar atrasados se a conta não tiver subscrições; a UI deve mostrar REAL-TIME, DELAYED ou FROZEN por símbolo.
- Paper e live em simultâneo podem exigir segundo username IBKR. Isto deve estar documentado.

9. Backtesting e simulação

Backtesting é uma peça central e deve surgir cedo no projeto. A app deve permitir simular uma estratégia em dados passados antes de pensar em ordens reais.

- Inputs: símbolo(s), timeframe, período, estratégia, parâmetros, capital inicial, comissão, slippage e modelo de execução.
- Outputs: equity curve, drawdown curve, lista de trades, métricas, exposição e comparação com buy-and-hold quando aplicável.
- Métricas mínimas: retorno total, CAGR, max drawdown, win rate, profit factor, Sharpe, average win/loss, perdas consecutivas, número de trades.
- A simulação deve guardar a configuração completa e a versão da estratégia para permitir reproduzir o resultado.
- Deve existir aviso visual quando há poucos trades, período curto, excesso de parâmetros ou possível overfitting.

10. RiskEngine

O RiskEngine é o travão central. Nenhuma ordem deve ser sequer apresentada como confirmável sem passar por esta camada.

Regra	Função
max_order_notional	Limita o valor máximo de uma ordem individual.
max_position_per_symbol	Impede concentração excessiva num único ativo.
max_open_positions	Limita número de posições abertas em simultâneo.
max_daily_realised_loss	Bloqueia novas ordens quando a perda realizada diária

	ultrapassa o limite.
require_stop_loss	Exige stop associado antes de permitir confirmação.
kill_switch_active	Bloqueia novas ordens e tenta cancelar ordens abertas.

11. Fluxos críticos

11.1 Fluxo de sinal

Market data -> IndicatorEngine -> StrategyEngine -> Signal -> Database -> Frontend

11.2 Fluxo de backtest

Historical bars -> IndicatorEngine -> StrategyEngine -> Broker simulator -> Metrics -> Report

11.3 Fluxo de ordem preparada

Signal escolhido ou ordem manual
 -> OrderPreparationService
 -> RiskEngine
 -> Confirmation payload
 -> Frontend modal
 -> confirmação humana
 -> IBKRAdapter.placeOrder
 -> broker order/fill/status
 -> audit_log

12. Endpoints API sugeridos

Método	Endpoint	Objetivo
GET	/health	Estado básico do backend.
GET	/mode	Modo atual: paper/live e estado do banner.
GET	/instruments	Lista de símbolos configurados.
GET	/market-data/{symbol}/bars	Velas OHLCV por símbolo/timeframe.
GET	/signals	Sinais gerados e filtros por símbolo/estratégia.
POST	/backtests	Criar execução de backtest.
GET	/backtests/{id}	Obter resultados e métricas.
POST	/orders/prepare	Preparar ordem a partir de sinal ou input manual.
POST	/orders/{id}/confirm	Confirmar uma ordem específica.
POST	/orders/{id}/reject	Rejeitar uma ordem preparada.
POST	/risk/kill	Ativar kill switch.
POST	/risk/clear-kill	Desbloquear após ação explícita.
GET	/audit	Consultar eventos de auditoria.
POST	/tax/export/{year}	Gerar exportação fiscal.

13. Testes e qualidade

- Unit tests para todos os indicadores com fixtures conhecidas.
- Unit tests para estratégias, garantindo que geram sinais esperados em datasets controlados.
- Testes de RiskEngine para cada limite e rejeição.
- Testes de backtesting com datasets pequenos onde o resultado esperado é conhecido.
- Testes de API para endpoints críticos.
- Testes de frontend para modal de confirmação, bloqueio por risco e kill switch.

- Testes de integração com IBKR paper quando o Gateway estiver disponível; mocks quando não estiver.
- Linting e formatação automáticos no backend e frontend.

14. Fases de implementação

Fase 1 - Scaffold e infraestrutura

Repo, backend FastAPI, frontend Vite, PostgreSQL via Docker Compose, SQLAlchemy, Alembic, logging, config e dashboard vazio.

Fase 2 - Dados históricos e modelo OHLCV

Importador CSV, tabela market_bars, endpoints de leitura e gráfico básico de candles.

Fase 3 - Indicadores

IndicatorEngine com SMA, EMA, RSI, MACD, Bollinger, ATR, VWAP; testes com fixtures.

Fase 4 - Estratégias e sinais

Strategy base class, 3-4 estratégias de referência, geração de sinais explicados e painel de sinais.

Fase 5 - Backtesting

Runner de simulação, broker_sim, comissões, slippage, métricas, equity curve, drawdown e persistência de runs.

Fase 6 - Integração IBKR paper para dados

IBKRAdapter, ligação ao Gateway paper, account summary, posições, histórico e fallback de data type.

Fase 7 - Streaming live/paper e dashboard operacional

Websocket/streaming, atualização de candles, estado da ligação, banners de dados atrasados.

Fase 8 - RiskEngine e kill switch

Limites server-side, rejeições explicadas, kill switch com bloqueio global e auditoria.

Fase 9 - Preparação e confirmação de ordens paper

Fluxo preparar -> rever -> confirmar, sem batch, com audit trail e envio apenas em paper.

Fase 10 - Exportação fiscal

CSV/XLSX anual com trades realizados, conversão EUR e fonte FX explícita.

Fase 11 - Guardrails live

Modo live apenas por configuração deliberada, banner persistente e ecrã de reconhecimento. Não implementar auto-trading.

15. Critérios de aceitação globais

- O projeto arranca localmente com comandos documentados.
- A base de dados é criada por migrações Alembic, não manualmente.
- Cada fase tem testes e só avança quando os testes passam.
- A mesma lógica de indicadores/estratégias é usada em backtest e em live/paper.
- Não há endpoint, botão ou flag que transmita ordens sem confirmação humana específica.
- O audit trail permite reconstruir o motivo de cada sinal e cada ordem.
- O modo paper/live está sempre visível.

- Falhas de broker, dados atrasados e reautenticação são visíveis no UI.

16. Estrutura de pastas sugerida

```
/backend
  /app
    /api
    /audit
    /backtesting
    /brokers
    /core
    /data
    /indicators
    /orders
    /risk
    /signals
    /strategies
    /tax
  /tests
  alembic/
/frontend
  /src
    /api
    /charts
    /components
    /pages
    /types
  /docker-compose.yml
  /README.md
  /SPEC_TECH.md
```

17. Notas de segurança e responsabilidade

Trading, especialmente day trading, envolve risco elevado. A app deve reforçar disciplina, auditoria e simulação, não criar ilusão de certeza. Qualquer resultado de backtest deve ser apresentado com caveats claros: desempenho passado não prevê resultados futuros.